Árbol de problemas

Problema central:

- El diseño y el control de los patrones de tejido en el telar es un proceso manual, sin registro digital ni forma de repetirlo con exactitud.

Causas:

1. El patrón de tejido se define a mano o de memoria. No hay una herramienta para diseñar visualmente ni para guardarlo.

2. No existe una base de datos ni un sistema central. La información se anota, queda en papel o en archivos sueltos.

3. El telar no está conectado a ningún sistema informático. La máquina y cualquier registro están totalmente separados.

4. El prototipo inicial guardaba todo en el navegador. Dependía de un único dispositivo, si se cambiaba de máquina o se borraba el caché, se perdía todo.

5. No hay trazabilidad de la producción. No queda registro de qué se tejió, cuándo, ni cuánto avanzó cada trabajo.

6. No hay control de acceso al sistema. Cualquiera puede usar la interfaz; no hay forma de identificar de manera segura quién opera o hace cambios.

7. El MVP presentado no incluía un login con una mínima capa de seguridad, y hacía falta. Cualquiera podría utilizar la página web sin problema alguno, lo cual deja vulnerables los datos de las matrices.

Efectos:

1. Se pierde tiempo reconfigurando dibujos que ya se habían hecho antes.

2. Es fácil cometer errores al cargar un dibujo a mano, y difícil detectarlos.

3. No se puede repetir un diseño exacto en otro momento o en otra máquina.

4. No hay historial para analizar la producción ni para auditar qué pasó.

5. El conocimiento depende de la persona, no del sistema: si esa persona no está, se complica.

6. Deja vulnerables los datos de las matrices, por lo cual, cualquiera con el link a la página web podría modificarlos en cualquier momento.

Árbol de soluciones

Objetivo central:

- Digitalizar el diseño y el control de los dibujos de tejido, con un sistema central que guarda todo y que puede conectarse al telar físico.

Medios resolutivos:

1. Se creó un editor visual de patrones (la página web): una cuadrícula donde cada fila lleva una sola bobina (en cada pasada entra una sola trama) y cada columna es una bobina de selección, con colores por fila. Resuelve la causa 1.

2. Se diseñó una base de datos central (PostgreSQL en Neon) con 4 tablas: patrones, telares, historial y errores (después se sumaron las del ingreso de usuarios). Resuelve las causas 2 y 5.

3. Se construyó un backend (Node.js + Express) que expone una API REST: es el intermediario único entre la interfaz, la base de datos y el telar. *Resuelve la causa 2 y habilita la 3.*

4. Se reemplazó el almacenamiento local en el navegador por la base de datos real: ahora cada acción se guarda en el servidor al instante, no en el navegador. *Resuelve la causa 4.*

5. Se desarrolló el firmware del ESP32 que conecta el telar físico al sistema, accionando los relés de Marcha, Pausa y Retroceder según lo que indica el backend y avisando cuando alguien usa la botonera a mano. *Resuelve la causa 3.*

6. Se registra el historial de producción y un log de errores en la base de datos. *Resuelve la causa 5.*

8. Se agregó un login por huella dactilar (con el estándar WebAuthn), para que el acceso al sistema sea seguro y personal. La huella nunca se guarda en el servidor: solo se almacena una clave pública para verificar la identidad. Resuelve las causas 6 y 7.

Fines:

1. Se ahorra tiempo: un dibujo guardado se reutiliza en segundos.

2. Se reducen los errores: el editor valida los datos y el sistema controla la consistencia.

3. Se puede repetir un diseño exacto las veces que haga falta, en el telar que sea (por ahora en uno solo; está planeado extenderlo al resto si funciona).

4. Queda trazabilidad completa: el historial registra qué, cuándo y cuánto se produce.

5. El conocimiento vive en el sistema, no en una sola persona. El programa está hecho para que sea lo más intuitivo posible y fácil de manejar para cualquier usuario (con o sin experiencia).

6. El acceso es seguro y trazable: cada trabajador entra con su huella y su usuario, sin contraseñas que se puedan compartir o perder. En caso de cambio de celular o huella inutilizable, hay un código de recuperación.

Cómo llegamos a la solución

1. Punto de partida: había un prototipo que solo funcionaba en el navegador (Mockup), con todo guardado en el almacenamiento local del navegador. Servía para dibujar, pero no era un sistema: sin servidor, sin base de datos, sin conexión a nada.

2. Primer problema detectado: para realizar el MVP, lo primero que había que corregir era que los datos se perdían. La solución fue montar un backend con base de datos. Se diseñó el modelo de 4 tablas (para el funcionamiento básico de la página; luego se agregarían las del login con biometría) y se construyó la API REST. La web se reconectó a la API en vez del almacenamiento del navegador.

3. Segundo problema: dónde alojar la base de datos. La solución fue utilizar PostgreSQL en Neon, en la nube, para no depender de un servidor propio ni de una sola máquina, tener respaldos y acceso remoto.

4. Tercer problema: el sistema tenía que poder repetir un dibujo sin errores, guardar de forma limpia y correcta varios dibujos y tener en cuenta varios detalles (por ejemplo: no poder editar una matriz en ejecución, no poder repetir nombres, encontrar la manera más práctica de dibujar las matrices). La mayor parte se resolvió sumando transacciones y bloqueo de filas en el backend, para que las operaciones críticas (asignar un dibujo, avanzar el tejido) sean seguras y consistentes. El resto de los detalles se resolvió identificando las inconsistencias con el testeo constante de la página.

5. Cuarto problema: todo esto seguía siendo software y el telar real no estaba conectado, así que no teníamos en claro cómo iba a funcionar el sistema con la máquina. La solución fue diseñar el ESP32 como gateway, que consulta el estado al backend y acciona los botones de Marcha, Pausa y Retroceder del telar mediante relés en paralelo, sin invadir la electrónica de la máquina y con comportamiento seguro ante fallas (fail-safe).

6. Quinto problema: tenía que ser cómodo de utilizar para los usuarios. La solución fue pensar la página web para el celular, con modo claro y oscuro, para operar desde el teléfono sin depender de una computadora.

Como resultado, tenemos un sistema donde el problema central queda medianamente resuelto: se diseña en la web, se guarda en la base de datos a través del backend, y (en la etapa final de hardware) se ejecuta en el telar real mediante el ESP32.

Función y resolución del proyecto y del ESP32

Qué función cumple el proyecto:

El proyecto digitaliza el diseño y el control de los patrones de tejido de un telar industrial. Permite que un operario diseñe un patrón en una página web (pensada para el celular), lo guarde en una base de datos central a través de un backend, y que esa información se use para controlar el telar físico. En vez de depender del papel, de la memoria o de un solo dispositivo, todo queda registrado en un sistema al que se puede acceder y que mantiene el historial de lo que se produjo.

En resumen, el sistema completo:

- Permite diseñar y guardar patrones de tejido de forma visual.
- Centraliza toda la información en una base de datos (patrones, telares, historial de producción y errores).
- Coordina, a través del backend, qué dibujo está tejiendo cada telar y en qué posición va la producción.
- Conecta ese sistema con la máquina real mediante el microcontrolador ESP32.

Qué función cumple el ESP32:

El ESP32 funciona como puente entre el sistema informático y el telar físico. Su rol es el de intermediario: no reemplaza la electrónica ni la lógica propia de la máquina, sino que la comanda desde afuera, como si alguien apretara los botones del telar a distancia.

Qué hace el ESP32, paso a paso:

1. Se conecta a la red Wi-Fi y le consulta al backend, cada pocos segundos, en qué estado debería estar el telar (tejiendo o detenido). Compara ese estado deseado con el de la consulta anterior.

2. Si detecta un cambio, acciona el relé correspondiente para reflejarlo en el telar real.

3. Si se corta la red o el backend no responde, no hace nada: el telar sigue gobernado por su botonera física, que nunca deja de funcionar.

4. Registra cualquier problema propio (por ejemplo, un fallo de conexión) en el log de errores del sistema.

Qué controla el ESP32, hasta ahora:

Controla el arranque, la pausa y el retroceso del telar. Para eso usa tres relés (interruptores electrónicos) conectados en paralelo con los botones físicos de Marcha, Pausa y Retroceder de la máquina. Además sensa (sin accionar) esos mismos tres botones mediante optoacopladores, de modo que si un operario los aprieta a mano, la página web refleja el estado real del telar.

Qué acciones le manda al telar:

• Marcha (arranque): cuando desde la página web se asigna un dibujo a un telar y este pasa al estado "tejiendo", el ESP32 envía un pulso al relé de Marcha, y la máquina arranca.

• Pausa (detención): cuando desde la web se detiene la producción, el ESP32 envía un pulso al relé de Pausa, y la máquina se detiene.

• Retroceder: cuando desde la web se toca ⏪ (con confirmación), el ESP32 envía un pulso al relé de Retroceder, y la máquina retrocede una pasada.

De esta forma, una acción tan simple como asignar un dibujo o tocar "Pausa" en el celular se traduce, a través del backend y del ESP32, en un arranque o una parada real de la máquina. El ESP32 se comunica siempre con el mismo backend y la misma API que usa la página web,

por lo que sumar el hardware no obliga a rediseñar el sistema: solo se agregó un nuevo "cliente" que, en vez de mostrar información en una pantalla, mueve un telar.

Como ya se mencionó, en esta etapa el ESP32 comanda el arranque, la pausa y el retroceso del telar. La medición de las pasadas físicas mediante el sensor inductivo del Bloque C forma parte del alcance de esta etapa: el sensor entrega un pulso por cada giro completo de un eje del telar, de modo que el avance registrado corresponde al movimiento real de la máquina.

Por qué usamos módulos de relé y no relés sueltos

En el Nivel 1, el ESP32 tiene que «apretar» tres botones de la botonera del telar (Marcha, Pausa y Retroceder), que trabajan con 24 V de alterna. Hay dos formas de hacerlo: comprar relés sueltos, por ejemplo el Songle SRD-05VDC-SL-C (el mismo relé que traen los módulos), y armar alrededor de cada uno el circuito que lo maneja; o usar módulos de relé, que traen ese circuito armado y probado de fábrica. Elegimos los módulos: uno de dos canales para Marcha y Pausa, y uno individual para Retroceder. Estas son las razones técnicas.

1. El ESP32 no puede mover la bobina de un relé por sí solo

La bobina del SRD-05VDC-SL-C está pensada para 5 V: tiene unos 70 Ω y consume alrededor de 71 mA (0,36 W). Para cerrar el contacto necesita como mínimo el 75 % de su tensión nominal, es decir, unos 3,75 V. Un pin del ESP32, en cambio, trabaja con lógica de 3,3 V, y el fabricante fija para cada pin una corriente máxima del orden de 40 mA, con valores de trabajo recomendados bastante menores.

- **Tensión:** 3,3 V no alcanzan los 3,75 V que el relé necesita para enganchar con seguridad.
- **Corriente:** 71 mA es casi el doble del máximo de un pin. Conectar la bobina directo dañaría la salida del ESP32 o lo haría reiniciarse.

Por eso un relé nunca se conecta directo al ESP32: hace falta una etapa intermedia que tome la corriente de los 5 V de la fuente y deje al pin solo la tarea de dar la orden.

2. Lo que habría que agregar alrededor de cada relé suelto

Para manejar un relé suelto desde el ESP32 hacen falta, por cada canal, como mínimo:

- **Un transistor NPN** (por ejemplo un BC547 o un 2N2222), que hace de llave para la corriente de la bobina.
- **Una resistencia en la base del transistor** (alrededor de 1 kΩ), que limita la corriente que sale del pin.
- **Un diodo de protección** (por ejemplo un 1N4007) en paralelo con la bobina y en sentido inverso. Cuando se corta la corriente, la bobina se comporta como un inductor y genera un pico de tensión inversa ($V = L \cdot di/dt$) que puede superar varias veces la tensión de alimentación. Sin el diodo, ese pico puede perforar el transistor (el BC547 soporta 45 V entre colector y emisor) y además mete ruido en la línea de 5 V que alimenta al ESP32.
- **Una resistencia que fije el estado del transistor** mientras el ESP32 arranca y, opcionalmente, un LED con su resistencia para ver el estado del relé.

En total son entre 4 y 6 componentes por canal: entre 12 y 18 piezas y más de 40 soldaduras para los tres relés. Ninguno de esos componentes está en la lista: habría que comprarlos, armarlos y verificarlos.

3. El módulo ya trae ese circuito resuelto

Cada canal de un módulo de relé trae soldado y probado de fábrica todo lo anterior:

- El transistor, el diodo de protección contra el pico de la bobina, las resistencias y el LED que indica el estado de cada canal.
- **En el módulo de dos canales, además, un optoacoplador por canal** (un 817, el mismo componente que usamos en el sensado). La entrada IN enciende un LED interno y el transistor del relé se

maneja con esa luz, sin conexión eléctrica directa con el pin del ESP32.

- Borneras de tornillo para los contactos (normal abierto, común y normal cerrado), dimensionadas para los 10 A del relé, y una tira de pines para las señales.
- En el de dos canales, el jumper JD-VCC, que permite alimentar las bobinas con una fuente separada si en el futuro hiciera falta aislar por completo el ESP32 del lado de los relés. Hoy va puesto y todo se alimenta con los mismos 5 V de la HLK-5M05.

4. Arranque seguro, con el firmware ya resuelto

El firmware y el conexionado están hechos y probados en función de los módulos. El de dos canales es activo en bajo: el relé se cierra cuando el ESP32 pone el pin en 0 V. Por eso IN1 e IN2 llevan una resistencia de 10 kΩ a 3,3 V (pull-up), que mantiene abiertos los relés de Marcha y Pausa mientras el ESP32 arranca y sus pines todavía no están configurados. El de Retroceder es activo en alto y lleva, por la misma razón, una resistencia de 10 kΩ a GND (pull-down). La polaridad de cada relé está configurada en el firmware (RELE_MARCHA_ACTIVO_BAJO, RELE_PAUSA_ACTIVO_BAJO y RELE_RETROCEDER_ACTIVO_BAJO, en config.h).

Con relés sueltos manejados por un transistor NPN, Marcha y Pausa pasarían a ser activos en alto. Si se dejara el mismo pull-up, el relé quedaría cerrado cada vez que el ESP32 se enciende o se reinicia: sería como apretar el botón de Marcha sin que nadie lo pida, y **el telar arrancaría solo**. Pasar a relés sueltos obligaría a rediseñar las resistencias de arranque, cambiar la configuración del firmware y repetir todas las pruebas.

5. Confiabilidad en el ambiente de la fábrica

El gabinete va montado junto a un telar que vibra durante toda la jornada, en un ambiente con pelusa textil. Cada soldadura hecha a mano es un punto posible de falla, por vibración o por una soldadura fría. El módulo concentra el circuito del relé en una placa fabricada en serie, con pistas y

separaciones de fábrica, y la plaqueta del proyecto queda con menos componentes y menos soldaduras.

Además, la parte que más ruido eléctrico genera, la bobina que engancha y suelta, queda en una placa aparte, separada de la plaqueta donde están el ESP32 y los optoacopladores del sensado, con su capacitor de 100 nF cerca de cada módulo.

6. Mantenimiento y reemplazo

Los módulos van atornillados sobre una plaqueta con separadores y tornillos M3, las señales llegan por cables dupont y los contactos por borneras. Si un relé se gasta o se daña, el módulo se cambia en pocos minutos con un destornillador, sin soldador y sin desarmar la plaqueta principal. Con un relé suelto habría que desoldar sus cinco patas y revisar el resto del circuito del canal.

7. Costo y disponibilidad

Los módulos ya se compraron y están en la lista de componentes del Bloque A. Los relés sueltos obligarían a sumar a la lista los transistores, diodos y resistencias de cada canal, además de las horas de armado y de prueba.

Comparación

Aspecto	Relé suelto	Módulo de relé
Manejo desde el ESP32	Necesita transistor, resistencia y diodo por canal	Ya los trae: el pin solo da la orden
Protección contra el pico de la bobina	Hay que agregar el diodo	Incluida
Aislamiento del ESP32	Ninguno, salvo que se agregue un optoacoplador	Un optoacoplador por canal en el módulo de dos canales

Aspecto	Relé suelto	Módulo de relé
Estado al encender el ESP32	Hay que rediseñar las resistencias y el firmware	Resuelto y probado (pull-up y pull-down)
Componentes extra en la plaqueta	Entre 12 y 18, con más de 40 soldaduras	Ninguno
Conexión de los contactos	Soldar cables a las patas o agregar borneras	Borneras de tornillo incluidas
Reemplazo en la fábrica	Desoldar y volver a soldar	Desatornillar y cambiar el módulo

Conclusión

El relé es el mismo en los dos casos: un Songle de 5 V y 10 A. Lo que cambia es todo lo que lo rodea. El módulo resuelve de fábrica la etapa que maneja la bobina, la protección contra su pico de tensión, el aislamiento y las borneras, y es lo que el firmware y el conexionado del proyecto ya tienen en cuenta. Por eso usamos módulos de relé, montados sobre la plaqueta, y no relés sueltos.

Niveles de control del telar (alcance del ESP32)

Esta sección aclara qué parte del telar controla el sistema hoy y qué quedaría como evolución para las siguientes etapas. El relevamiento confirmó que el telar de implementación (Vamatex C 201) no es un Jacquard de tarjetas sino un telar con cinta de papel perforada y cuatro bobinas de selección; las explicaciones de abajo conservan el análisis inicial, y la vía que se sigue es la del control por marcos del final.

El telar tiene, en la práctica, dos "máquinas":

En este tipo de telar (tejido plano con mecanismo Jacquard) conviven dos funciones bien separadas:

1. La máquina de accionamiento. Es la que mueve el telar: arranca, para, avanza y retrocede el tejido. Se opera con la botonera física (Marcha / Pausa / Retroceder).

2. La máquina lectora de secuencia (el Jacquard con tarjetas perforadas). Es la que "lee" el dibujo y le dicta al telar, pasada por pasada, qué hacer para formar la matriz. Hoy funciona con tarjetas perforadas físicas: aproximadamente una tarjeta por ciclo, y cambiar de diseño implica desmontar e intercambiar cientos de tarjetas a mano (un proceso largo y lento).

El sistema de este proyecto apunta a reemplazar la función de esa segunda máquina (la lectora) con algo digital: la página web y la base de datos pasan a ser las que definen y almacenan la secuencia del patrón, en vez de las tarjetas de cartón.

Los dos niveles de control:

Conviene separar el control del telar en dos niveles, porque tienen muy distinta dificultad y conviene ser honesto sobre cada uno.

Nivel 1, Arranque y parada (lo que el ESP32 hace hoy):

El ESP32 acciona la máquina de accionamiento: darle Play (Marcha), Pausa y Retroceder al telar, mediante los tres relés en paralelo con la botonera.

- Estado: instalado y probado en el telar el 19/09/2026.

- La dificultad de este es más baja, ya que no invade la electrónica del telar; solo "aprieta los botones" a distancia, y la botonera manual sigue funcionando igual.

- Permite manejar el telar desde la web (darle play, pausarlo) sin estar físicamente frente a la máquina.

Este nivel ya está instalado en el telar y no tiene inconvenientes de fondo.

Nivel 2, Dictar la secuencia del patrón (instalado y probado):

Reemplazar a la máquina lectora significa que el sistema no solo dé Marcha, sino que le entregue al telar la secuencia completa del dibujo (qué hacer en cada pasada), ocupando el lugar de las tarjetas perforadas (en este telar, la cinta de papel).

- Estado: instalado y probado en el telar el 28/10/2026, con el gabinete y los cuatro canales: el sensor cuenta las pasadas y cada relé LCA110 selecciona su bobina según el dibujo cargado en la aplicación. Antes se había probado en protoboard (10/10/2026) y con el primer canal en la máquina (17/10/2026).

- El punto a resolver era que la lectura parecía un mecanismo Jacquard por tarjetas perforadas físicas: un sistema mecánico, no una señal eléctrica simple. El relevamiento mostró que en este telar la secuencia la lee un lector óptico sobre una cinta de papel, que da una señal eléctrica por bobina, y eso simplifica el reemplazo.

- Llegar al “Nivel 2” implica comandar las bobinas de selección que el telar ya tiene, para que reciban la secuencia desde el backend en lugar de leerla de la cinta de papel. Eso es un proyecto de hardware en sí mismo.

En resumen:

Ya se sabe que el sistema define y guarda los patrones (reemplaza la función de la máquina lectora a nivel de diseño y datos), y el ESP32

controla el arranque y la parada del telar (Nivel 1). Con esto, darle Play en la web y que el telar arranque es viable y sin inconvenientes de fondo.

Como mejora, el paso que sigue es que el ESP32 le dicte la secuencia completa al telar, reemplazando a la cinta perforada. En este telar no hace falta intervenir la mecánica: alcanza con un relé LCA110 en el lugar de cada lector óptico de la cinta, y es lo que se está desarrollando.

La solución viable para el Nivel 2: control por marcos:

Como los tejidos de esta fábrica son mayormente dibujos simples y repetitivos (por ejemplo, una cortina donde un dibujo se repite constantemente), y no logos ni figuras complejas, el “Nivel 2” deja de ser una meta lejana y se vuelve algo más alcanzable como prototipo real.

La idea clave: no hace falta control hilo por hilo:

El mecanismo Jacquard original controla cada hilo por separado (por eso puede hacer dibujos complejos, pero necesita cientos o miles de elementos). Un dibujo que se repite no necesita esa complejidad: se puede tejer controlando marcos (grupos de hilos que suben y bajan juntos), que es justamente lo que hace un telar dobby.

Toda tela con un patrón que se repite (cortinas, rayas, espigado, panal, etc.) se define por una secuencia corta de combinaciones de marcos que se repite en bucle. Y esa secuencia corta que se repite es exactamente lo que el sistema ya hace hoy con la matriz de pasadas.

Cómo funciona:

En cada pasada del telar, algunos marcos suben y otros bajan; esa combinación forma el cruce de los hilos que da el dibujo. En este sistema, cada pasada se reduce a indicar qué bobina de selección se activa: en cada pasada entra una sola trama. En este telar son cuatro bobinas,

confirmadas por el dueño el 19/09/26; el sistema no fija esa cantidad y se adapta a otras configuraciones. El patrón repetitivo es una lista corta de bobinas, cada una tantas pasadas seguidas como indique su fila, que se repite en bucle. El backend le entrega esa secuencia al ESP32, y el ESP32 activa en cada pasada la bobina que corresponde.

El hardware necesario:

- Las bobinas de selección. El telar ya las tiene instaladas y funcionando: trabajan con 24 V en alterna y son las que hoy accionan las palancas leyendo la cinta de papel.

- El ESP32 recibe la secuencia del backend y, en cada pasada, activa la bobina que corresponde a través del relé LCA110 de su lector óptico.

- Como son pocos, entran directamente en los pines del ESP32: con cuatro canales sobran los GPIO libres, sin necesidad de ningún componente intermedio.

- No hace falta una fuente aparte: las bobinas siguen alimentadas por el telar con sus 24 V en alterna, y el relé de cada lector solo conmuta la señal.

Cómo se conecta con lo que ya está hecho:

- El editor ya define una matriz de pasadas que se repite. Cada fila de esa matriz indica qué bobina se activa y cuántas pasadas seguidas se teje.

- El backend ya guarda y entrega esa secuencia por la API; no hay que rediseñarlo.

- El firmware del ESP32 pasaría de accionar 3 relés (Marcha/Pausa/Retroceder) a accionar además los 4 relés LCA110 de los lectores ópticos, uno por cada una de las cuatro bobinas de selección del telar (confirmadas por el dueño el 19/09/26), en la misma placa.

En otras palabras: la parte de software ya está casi lista; lo que se suma es la parte física: un relé por lector óptico.

Conclusión:

Esto aplica si el telar es de control por marcos. El relevamiento lo confirmó: el C 201 es un dobby que lee una cinta de papel perforada y selecciona con cuatro bobinas, así que el retrofit es directo. En cualquier caso, para el tipo de tejido repetitivo de esta fábrica, el control por marcos es la vía correcta y está al alcance de un prototipo con ESP32.